

μInference

A verifiable minimum inference engine for LLMs

692 lines of trusted code.

Same output on seven different systems, bit for bit.

Runs on seL4, a formally verified microkernel.

What it is

A small LLM inference engine built to be checked.

Small

692 lines of code do the inference.
No libc. No libm. No heap.
No threads.

Reproducible

Same weights and prompt
give the same
logits on every system
tested.

Isolated

Runs as one seL4
protection domain.
No device access. No Linux.

Bounded

Memory use is fixed at build
time.
4.1 MB. No input can
change it.

What it is for

To check that some other system ran a model correctly, you have to run it again yourself.

That means you need an engine you trust more than the one you are checking.

Existing engines are too large to trust and do not give the same answer twice.

Why other engines are not reproducible

The cause is batching, not floating-point error.

vLLM, SGLang

They batch requests together.

Batch size changes how sums are split up.

Batch size depends on how busy the server is.

1000 runs of one prompt gave 80 different answers.

Forcing them to be reproducible costs 1.6-2x speed.

μInference

It does not batch.

One request at a time, one sum order, always.

So the problem does not arise.

Reproducibility costs nothing here.

It is a result of the design.

```
for (j = 0; j < n; j++) val += wr[j] * x[j];
```

This loop must stay exactly this shape.

Which maths operations are used

The IEEE-754 standard requires exact results for some operations, not others.

REQUIRED TO BE EXACT

$+$ $-$ $\times$ $\div$ $\sqrt{}$

Any two correct systems must agree.

NOT REQUIRED TO BE EXACT

`exp` `log` `sin` `cos` `pow`

Results differ between systems.

So the engine only uses the exact ones. The rest were replaced:

<code>expf</code>	used by softmax and SwiGLU	replaced with a polynomial	within 1 ULP of libm
<code>powf</code> <code>cosf</code> <code>sinf</code>	used by RoPE	replaced with a lookup table	table is hashed and checked
<code>sqrtf</code>	used by RMSNorm	kept, it is exact	no change needed
<code>malloc</code> , <code>OpenMP</code> , <code>RNG</code>	not maths, but not reproducible	removed	fixed memory, one thread

How the repo is organised

One engine. Four ways to run it. The engine code is the same in all four.

mucore/ — the engine

<code>mu_math.h</code>	85	the maths
<code>mu_core.h</code>	133	the interface
<code>mu_core.c</code>	474	the transformer

**692
lines**

hosts/posix

Runs on macOS or Linux.
For development.

159 lines

hosts/baremetal

Runs on ARM with no
operating system.

338 lines

hosts/x86_64

Runs on Intel with no
operating system.

365 lines

hosts/microkit

Runs on seL4 as one
protection domain.

157 lines

The model weights are built into each image. One hash covers the code and the weights.

Result: seven systems, one hash

Hash of the raw output numbers from every step. 3,072,000 bytes.

9b78b92a305dc59611b5c53a04b538ae9c4ae18aea6c1fdbf6e45e9848b23bd2

✓	clang 21, -O2	macOS	
✓	clang 21, -O0	macOS	different optimisation level
✓	clang 22, -O2	macOS	different compiler version
✓	GCC 16, -O2	macOS	different compiler
✓	clang 18, -O2	Linux	different OS and CPU (CI)
✓	ARM, no OS	QEMU	no C library at all
✓	Intel, no OS	QEMU	different CPU maths implementation
✓	seL4 / Microkit	QEMU	the target system

The test can fail

Built the wrong way, the output changes. So the test is real.

CORRECT BUILD

```
-fno-fast-math  
-ffp-contract=off
```

0 fused multiply-add

9b78b92a...

WRONG BUILD

```
-Ofast  
-march=native
```

96 fused multiply-add

351ebd7b... different

Why fused multiply-add changes the answer

$a*b+c$ as one instruction rounds once. As two instructions it rounds twice.

Comparing the text would not catch this. Comparing the numbers catches one bit.

Numbers

All measured.

692

lines of engine code

llama2.c is 973.
vLLM stack is millions.

4.13 MB

memory used

Same on all seven systems.
Fixed at build time.

161 tok/s

one CPU thread

Apple Silicon.
No BLAS, no SIMD, no threads.

1 ULP

worst maths error

Compared against libm
at 16 million points.

Compared against the original llama2.c

5 out of 5 test prompts give exactly the same text.

Replacing the maths functions did not change the output.

What is not claimed

Not tested on real hardware

Only in QEMU, an emulator.

Builds are not reproducible

The output is the same. The binary files are not identical.

Only one model tested

stories15M, 15 million parameters. Not tested at 7 billion.

No temperature sampling

Only greedy selection. Random sampling needs a fixed random seed.

No GPU support

A GPU needs signed vendor firmware and a closed compiler. Both would break the claim.

seL4 is not fully verified on ARM

Correctness and integrity are proven. Confidentiality is not. Single core only. No IOMMU.

Goals

DONE

- 692-line engine, four hosts
- Same output on seven systems
- Runs on seL4
- Tests pass in CI
- Matches llama2.c output

NEXT — MONTHS

- Prove there are no memory errors
- Prove the maths error bound
- Check reproducibility automatically
- Run on real hardware
- Test a larger model

LATER — RESEARCH

- Prove reproducibility as a theorem
- Prove the total error bound
- Use it to check other systems
- Move to RISC-V for stronger proofs
- Try integers instead of floats

Testing is not proving

Tests check some inputs. A proof covers all of them.

NOW

7 systems, 1 prompt, 24 steps.

A good sample.

Still only a sample.

GOAL

All inputs. All correct systems.

Checked by machine, once.

Covers inputs never run.

One thing proofs cannot do

A proof says the program is correct for all inputs. It says nothing about one specific run.

To show what happened on a specific run you need attestation or re-running.

Proofs make the engine worth trusting. They are not a receipt.

What to prove

Five parts, cheapest first. Each one is a separate, checkable claim.

S1	No memory errors or crashes, for any input	Frama-C	no proof work
S2	Memory use never exceeds the fixed limit	Frama-C	no proof work
S3a	Only the exact maths operations are used	clang plugin	no proof work
S3b	Output depends only on the inputs	CompCert	no proof work
S4a	The exp function is within a known error	Gappa	proof generated
S4b	The matrix multiply error bound	LAProof	some proof work
S4c	The total error for one token	combine S4a, S4b	proof work
S5	It computes a transformer correctly	Coq	large project

S1 to S4a need no hand-written proofs. Tools do the work.

Plan

Each step is a separate piece of work with its own test in CI.

0	Run CBMC to look for crashes	3 days	find problems early
1	Prove no memory errors (S1, S2)	3–6 weeks	no crashes, any input
2	Add the maths-operations check (S3a)	1 week	checked automatically
3	Prove the exp error bound (S4a)	2–4 weeks	known maths error
4	Prove the matrix multiply bound (S4b)	1–2 months	known matmul error
5	Build with CompCert (S3b)	2–4 weeks	reproducibility proven
6	Combine the error bounds (S4c)	months	total error known
7	Prove full correctness (S5)	research	not scheduled

Steps 0 to 3 take about two months and use existing tools.

AI cannot write the proofs yet

Measured by VERINA. Best model tested was o3, writing Lean.



So do not plan on it

A 5% success rate cannot be the main path.

What the plan does

Use tools that prove things automatically. Only about 100 lines need real proof work.

Next steps

1

Approve steps 0 to 3

About two months. Existing tools. No hand-written proofs. Result: no memory errors, and a known maths error bound.

2

Decide whether to do S5

Full correctness needs a reference transformer written in Coq. That is a large project on its own. Suggestion: leave it out for now.

3

Contact Theorem Labs

A research lab working on AI for formal verification. \$6M funding. Their tools found a bug in Anthropic's sampling code that testing missed.

github.com/luiscosio/muInference · PR #1